

AT&T Spam Detector

Détection automatique de SMS spam par Deep Learning & Transfer Learning

Marine Deldicque · Jedha Fullstack Data Science · Certification CDSD · Bloc 4 Deep Learning

5 571

SMS labélisés

2

modèles comparés

98.7%

accuracy best

01 Contexte & EDA

02 Pipeline NLP

03 Modèles

04 Résultats & Conclusion

01 · Contexte & EDA

Le problème

AT&T reçoit des millions de SMS par jour. Les spams exposent les utilisateurs à des arnaques et dégradent l'expérience.

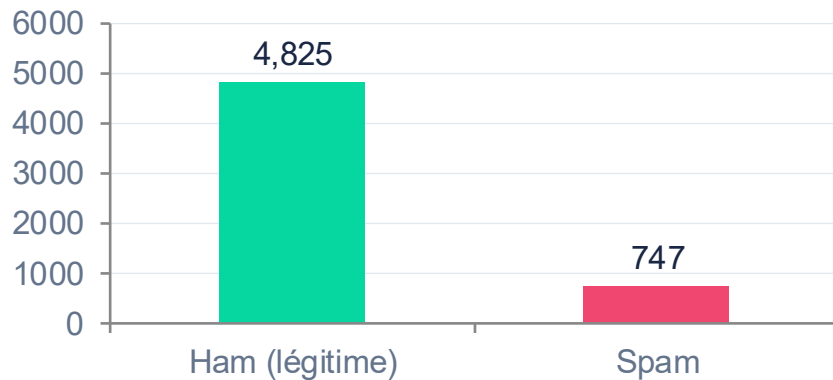
Objectif : classifier chaque SMS comme ham (légitime) ou spam à partir du texte uniquement, données non structurées.

Dataset 5 571 SMS labélisés (spam.csv)

Tâche Classification binaire : ham=0 / spam=1

Métrique clé F1 spam — l'accuracy globale est biaisée

Distribution des classes



86% ham / 14% spam → déséquilibre modéré

Spams : messages plus courts en moyenne

→ `class_weight` appliqué pour compenser

02 · Pipeline NLP : Prétraitement du texte

1 Nettoyage

Minuscules · suppression
URLs, chiffres, ponctuation

2 Tokenisation

Chaque mot → ID
numérique unique
Vocabulaire : 10 000 mots ·
OOV token

3 Padding

Tous les SMS à longueur
fixe
150 tokens (post-padding)

4 Split stratifié

80% train / 20% test
Stratify conserve le ratio
86/14

Exemple concret

SMS original

"FREE WIN PRIZE! Call
now 07777 999"

Après nettoyage

"free win prize call
now"

Après tokenisation

[45, 302, 891, 77, 204]

Après padding

[45, 302, 891, 77, 204,
0, 0 ... 0] (150 tokens)



Le tokenizer est fitté uniquement sur X_{train} : garantit l'absence de data leakage, même en NLP.

03 · Deux modèles entraînés

Modèle 1 Baseline

Embedding + GlobalAvgPooling + Dense

Embedding 10k mots → 32 dims

GlobalAveragePooling1D réduit la séquence en vecteur

Dense 64 (ReLU) + Dropout 30%

Dense 1 (Sigmoid) proba spam $\in [0, 1]$

 ~10 sec

CPU

9 epochs

EarlyStopping

class_weight

déséquilibre

Modèle 2 Transfer Learning

DistilBERT (HuggingFace) + Fine-tuning

DistilBERT pré-entraîné 66M paramètres

Entraîné sur Wikipedia+livres milliards de tokens

Fine-tuning : 2 epochs sur nos 5 571 SMS

Dense 2 classes ham / spam

 ~1 min

GPU T4

2 epochs

fine-tuning

AdamW lr=2e-5

optimiseur

04 · Résultats & Comparaison

Accuracy / Baseline

97.9%

F1 spam / Baseline

0.919

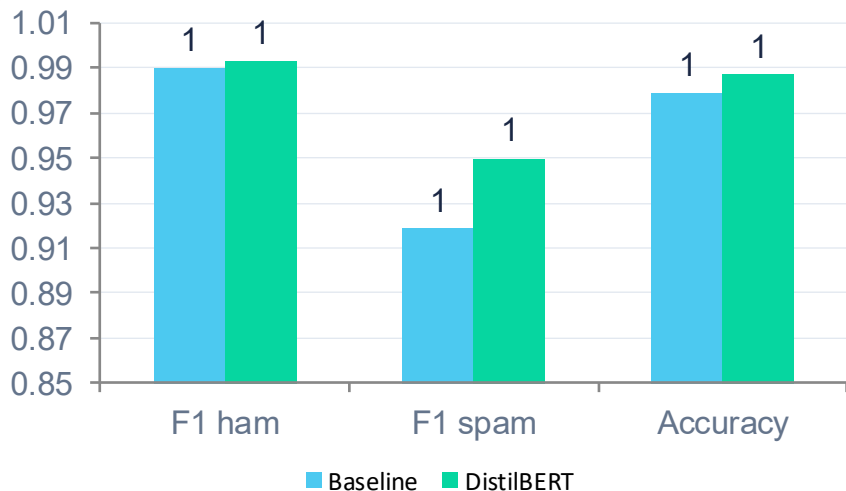
Accuracy / DistilBERT

98.7%

F1 spam / DistilBERT

0.950

Comparaison F1-score par classe



Matrices de confusion (test set)

	Prédit ham	Prédit spam
Réel ham	955	10
Baseline		
Réel spam	13	137
Réel ham	962	3
DistilBE		
RT		
Réel spam	5	145



Métrique prioritaire : F1 spam DistilBERT détecte 18 spams supplémentaires sur le jeu de test (145 vs 137)

04 · Conclusion & recommandation



EDA : dataset déséquilibré (86/14%) → class_weight appliqué, F1 spam choisi comme métrique principale



Pipeline NLP : nettoyage → tokenisation (fit sur train uniquement) → padding → split stratifié



Modèle 1 : Embedding + Dense : Accuracy 97.9% · F1 spam 0.919 · ~10 sec · CPU



Modèle 2 : DistilBERT (Transfer Learning) : Accuracy 98.7% · F1 spam 0.950 · +3.1 pts · ~1 min GPU



Recommandation : DistilBERT en production, détecte 18 spams supplémentaires. Le gain justifie le coût computationnel.